AI-Reconstruct: An Expandable AI-Accelerated Successor to Reconstruct

Abstract

With the advancement of Artificial Intelligence (AI), the acceleration of image processing becomes feasible. The efficiency of Reconstruct, an image annotation tool, has always been the issue for different annotation applications in many different fields. We added several modules to Reconstruct software architecture to facilitate the integration of AI models into Reconstruct. We used a cell segmentation task, specifically for the Giant Vacuole (GV) cell, as an example to demonstrate how to use the new software architecture to design and integrate an AI model into Reconstruct.

1. Introduction

Reconstruct is one of the first open-source image annotation applications. It provides a large set of annotation features which can be customized easily. Reconstruct has been applied to many different fields, such as neuroscience [1 – 7] to plant biology [8-9], entomology [10 – 12], human anatomy [13], herpetology [14], and materials science [15] , and even more. The widespread use of Reconstruct is because of the emerging demand for image annotation tools, plus its simple user interface, and plenty annotation options.

Since its release, additional annotation applications have emerged that are built on top of Reconstruct but integrate modern technologies to allow rapid imaging and automated segmentation. It gives rise a challenge that the architecture of the annotation application needs to be expandable so that modern technologies can be integrated into it seamlessly.

With the development of artificial intelligence (AI), especially deep learning (DL), new techniques in image processing has been multiplied. They can be applied to image enhancement, image segmentation, instance segmentation, image manipulation, image classification, etc. The application of these techniques can effectively improve the precision and efficiency of image annotation. In addition to the advanced techniques, we also need to add fundamental functionality to Reconstruct to make it compatible with the modern technology, such as resizing, augmentation, scaling, cropping, classification, enhancement, etc.

To add advanced features to Reconstruct, several fundamental challenges persist. Tasks like these require: (1) considerable programming expertise to access full annotation details and advanced features, (2) comprehension of the software architecture as a whole to make modification for the new functionality (3) software design expertise to properly integrate new functionality.

The success of integration depends on several key factors: (1) maintain existing functionality, (2) manage the consistency of future expansions, (3) ease the addition of the new functionality. The total workload is almost equal to rewrite the entire Reconstruct, even if only a few features need to be added. At this point, we need to systematically modify software architecture of Reconstruct to make it easy to integrate advanced features through programming.

The sections below are organized as the following. Section 2 introduces the technical route of Reconstruct. Section 3 shows the new architecture which can facilitate the integration of new features, especially in deep learning. Section 4 uses an example in image segmentation to show how to add a

Boundary Tracing feature to Reconstruct. In section 5, we draw the conclusion.

2. Technical Route of Reconstruct

In Reconstruct, a series is a project file that manages the entire project. The series file stores information that spans multiple sections, such as series metadata, object attributes, current alignment, user information, and etc. Each section stores information in a separate, JSON-structured file that includes annotation data for every object. Although sections and images can map to each other one-on-one, image and section files can be stored in different locations, because section files have the location information of the corresponding images. The hierarchical structure is shown in Figure 1. Once a series is loaded into Reconstruct, all the detailed information of sections, objects, etc. are loaded.

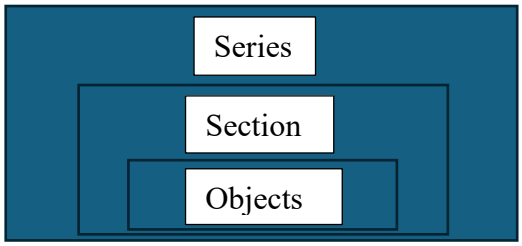


Figure 1. Hierarchical Structure of Reconstruct Elements.

When a series is loaded and the entire annotation journey starts, the first image shows in the main window, which is also the first section in the series, as shown in Figure 2 Step 1. In the main window, a target cell can be spotted in the image, as shown in Figure 2 Step 2. Reconstruct can automatically align all the image. By default, the coordinates of the same cell in difference sections can be within the same range in Reconstruct, although the shape of the cell in each section of the cell looks different because they are different slices of the same cell.

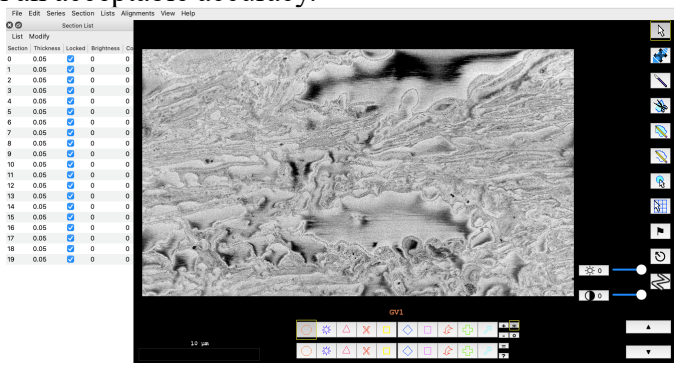
We also need to find all the sections that the cell appears, as shown in Figure 2 Step 3. After locating the cell in one image, we can fix the coordination to the position of the cell. In other sections, the images of the same cell are all in the same location. Normally, an image is a snapshot of one slice of the 3D object, a set of images in adjacent sections belong to the same 3D object. We name the same object in different sections with the same object name.

Traces for a cell can be defined by drawing the cell boundaries, as shown in Figure 2 Step 4. To segment a cell in different sections, we annotate the cell boundary in every image it appears. Before we start drawing boundaries, we can select a stamp from the list of stamps on the bottom of the main widow. The stamp can be customized to change the stamp shape, color, name, tag, etc. Because we normally define a stamp for each object, it is a convention that we use the object name to name the stamp.

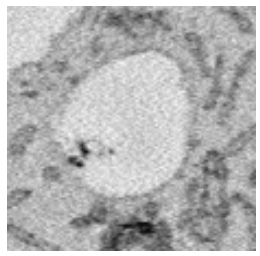
After all the cell boundaries are segmented in different sections, the 3D structure of the cell can be defined, as shown in Figure 2 Step 5. A 3D scene is generated by linking the 2D traces. After all the boundaries of a 3D object in different sections are circled with the stamp, we can add the object to a 3D scene and the 3D view of the object can appear in the 3D scene window.

In this workflow, it is time-consuming to draw boundaries. The typical speed of tracing a cell is around 10 hour. Given a large volume of cells, it is not feasible to manually complete cell tracing within the time frame of diagnoses/treatments. The way to accelerate this process is through Artificial Intelligence

(AI) that we train a model with the tracing task in advance so that the model can automatically complete cell tracing with an acceptable accuracy.



Step 1. Load Images

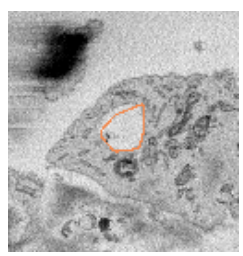


Step 2. Locate a cell in a Image

List Filter Find Groups Columns

Name	Start	End	Host	Groups	Locked
GV1	1	7			☐

Step 2. Find the Start and the End Sections of the cell



Step 3. Draw the Boundary of the Cell in Each Section

Step 4. Generate 3D structure for the Cell

Figure 2. The Overall Workflow of Reconstruct.

3. Deep Learning Integration

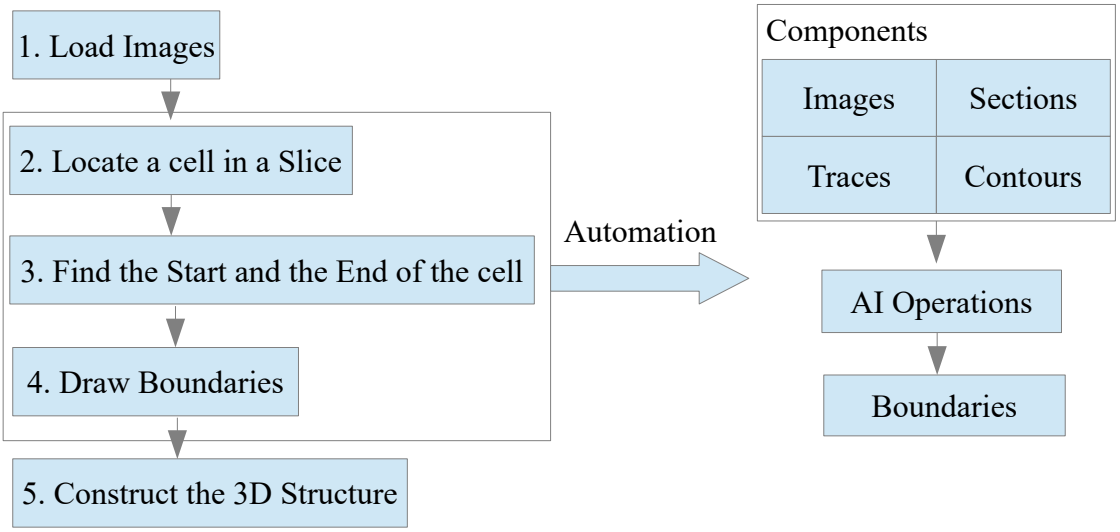


Figure 3. Automation Workflow

To automate the process of image annotation, we need to prepare several components to operate the functionality, such as images, sections, traces and contours, as shown in Figure 3. The results of the operations are the automatically generated boundaries which are the image annotations. These components are defined in Reconstruct. We use these existing components to build a deep learning wrapper so that, when a new functionality needs to be added, it only needs to call the wrapper to pull out the attributes, as shown in Figure 4, and functions of the components.

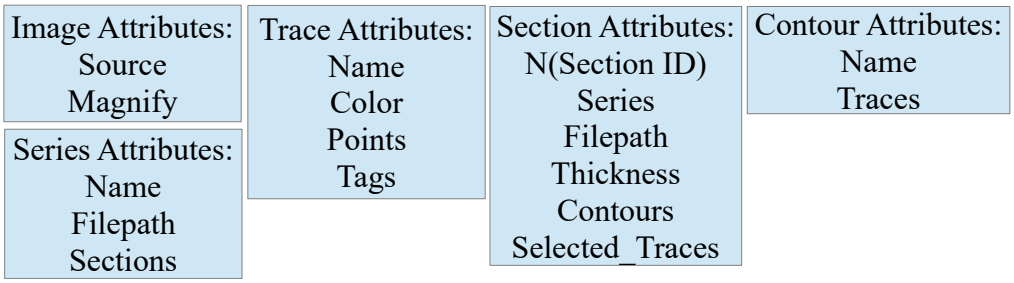


Figure 4. Component-Attribute Diagram

As shown in Figure 5, in order to operate these components, information passing between components trigger different actions to complete specific tasks. We use a cell segmentation task as an example to show how to manage these actions in deep learning projects and also to prove both components and actions are suitable to initiate any deep learning tasks.

When a cell is spotted in a section, a Region Of Interest (ROI) can be annotated quickly. However, within the ROI, the tight boundary is the trace of the cell which requires domain-knowledge to annotate it correctly. Pre-trained DL model can be used to automatically convert the ROI to the tight boundary with high-quality results. Figure 6 uses a swimming lane diagram to demonstrate how the automatic segmentation can be implemented through the following steps. In the next section, we use an example to explain the implementation details of this task.

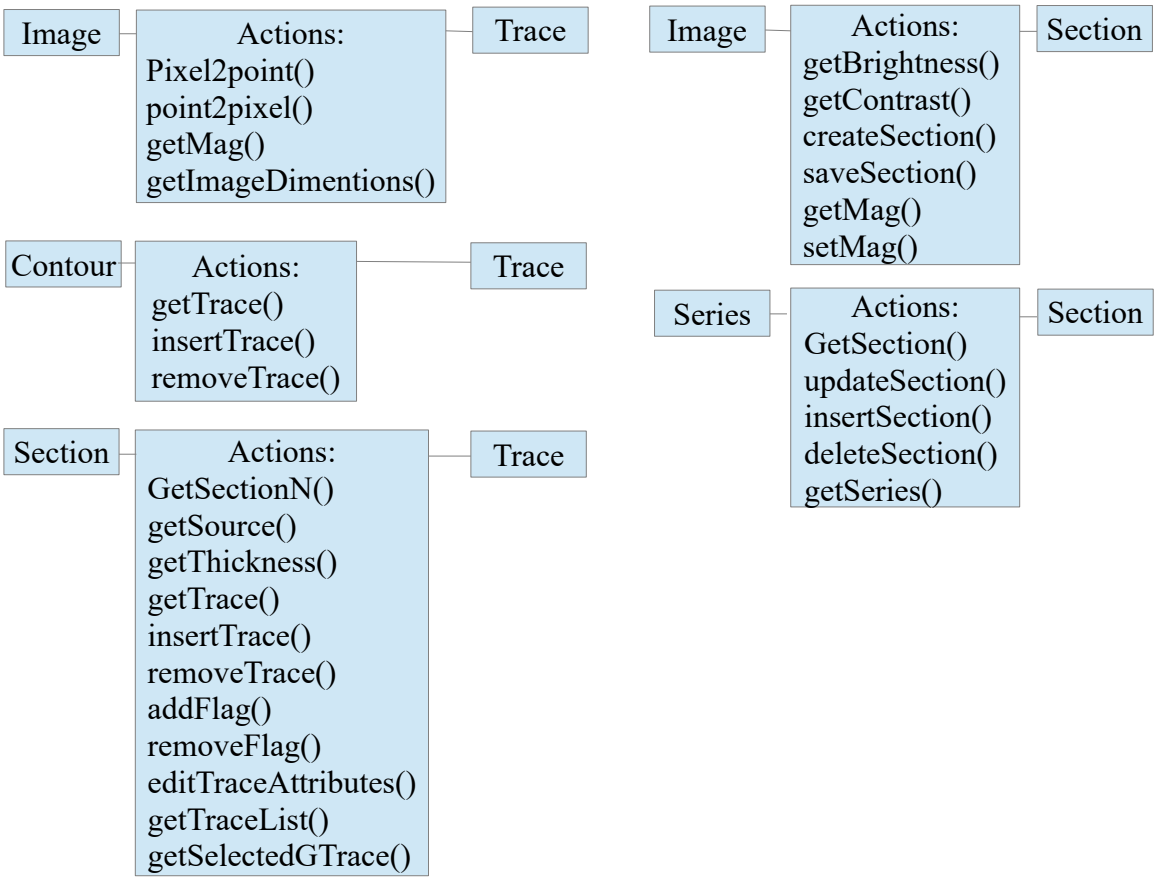


Figure 5. Component-Action Diagram

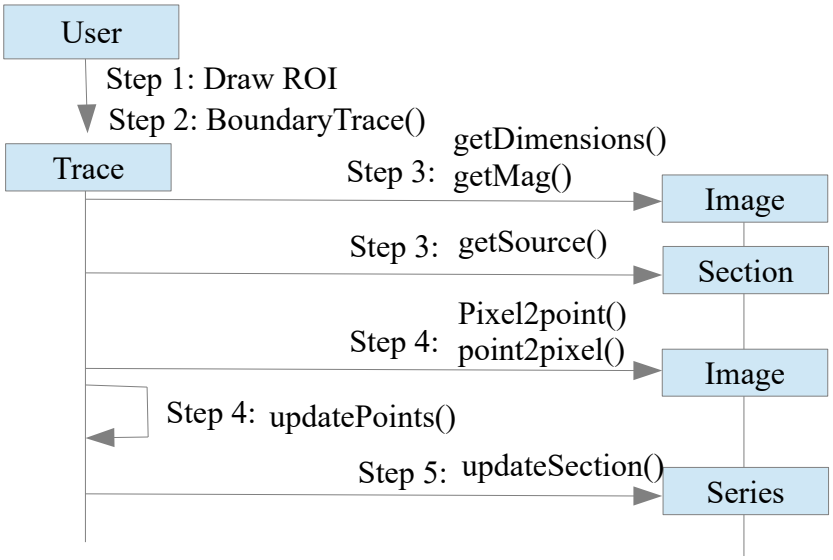


Figure 6. Swimming Lane Diagram for Image Segmentation

- Step 1. Draw ROI
- Step 2. Call “Boundary Trace” function to trigger DL model

- Step 3. Trace component load the source file. Trace component reads the source image by calling getSource(), read image magnification by calling getMag(), read image dimensions by calling getImageDimensions().
- Step 4. Trace component modifies points. Trace component segments the image by using the points defined for the ROI. Because the original image uses pixels as the computation unit and trace points use points as computation units, pixel2point() and point2pixel() functions can be used to make sure the units of the RIO and the image are consistent. After the DL model makes predictions with the ROI segmentation, updated boundary points can be returned. Those points can be used to define the tight boundary.
- Step 5. Trace component updates section component. After the trace points are modified, updateSection() is called to save changes to the section component.

4. Boundary Tracing through Deep Learning

Primary Open-Angle Glaucoma (POAG) is called the silent thief of vision and is one of the leading causes of irreversible blindness worldwide. Aqueous outflow resistance can lead to intraocular pressure (IOP) which is a major risk factor of Primary Open-Angle Glaucoma (POAG). However, the mechanism for developing the resistance is unclear but is determined by the physical surroundings of the Inner Wall (IW) of Schlemm's Canal (SC).

GVs can form due to the pressure change between anterior chamber and episcleral veins [16-18]. GVs with both basal openings and luminal I-pores provide a channel for AH to pass into SC from the JCT. Two previous Scanning Electron Microscopy (SEM) studies [19, 20] have shown a decrease in the number of GVs with pores in eyes with POAG.

To measure the accurate percentage of GVs with pores and openings, a large amount of serial thin sections must be examined which can provide in-depth view of the sample area for the POAG analysis but is labor intensive. Serial Block-Face Scanning Electron Microscopy (SBF-SEM) can provide serial cross-sectional images for the structure of the outflow pathway which can be used to reconstruct the 3D structure pathway.

From 2D SBF-SEM images to 3D pathway structures, GV volume can be generated in Reconstruct , as shown in Figure 2, by tracing each serial segmentation of a GV by hand and multiply it by the thickness of each image. Normally, it takes 40 hours to trace one to three GVs, and at lest 200 samples are required for each study. It gives rise to a question if we can develop a faster method for 3D cell construction to make it feasible to perform a cell study.

The cell construction process can be accelerated by using pre-trained deep learning models. We use GV reconstruction application as an example to show how to load a deep learning model into Reconstruct, automate the boundary tracing with Reconstruct, and eventually generate the 3D structure for a Giant Vacuole cell.

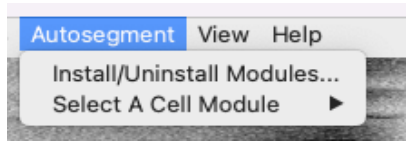
In the section below, we describe module installation, annotation interface, object organization, automatic segmentation, ROI alignment, and data visualization.

4.1 Module Installation

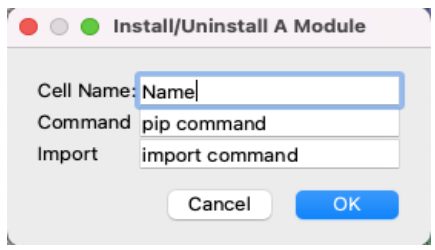
Each cell type requires a pre-trained models to trace boundaries. The pre-trained models can be published to the Python Package Index (PyPI) and can be installed into Reconstruct by using the

Python installer – pip. Figure 7 shows how to install/uninstall a deep learning package, how to define a connection string to import a package into Reconstruct, and how to select a cell type to perform automatic boundary tracing.

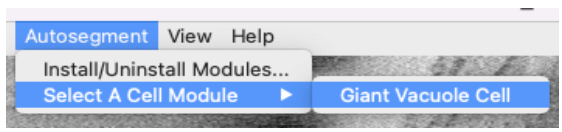
All the packages are managed by a distribution file. It records the cell names, the Python packages, and the module importing strings. As shown in Figure 8, All the models follow the same syntax to conduct prediction: the input is the ROI of the image, the predict statement is called to conduct prediction for each instance of a module, and the output is the contour of the boundaries which can replace the coordinates of the ROI and define the boundaries of the cell.



Step 1. "Autosegment" Menu Item



Step 2. Install/Uninstall a Module



Step 3. Select a Cell for Segmentation

Figure 7. Deep Learning Module Integration

```
Pseudo-Code

Import GiantVacuoleModel

Function GiantVacuoleSegmentation(ROI):
        # initiate an instance for a cell model
        model = GiantVacuoleModel()

        # generate the contour of the cell by using the ROI
        output = model.predict(ROI)

        return output
```

Figure 8. Pseudo Code for Predicting Cell Boundaries.

4.2 Object Organization

Automatic segmentation is conducted in the backend and directly works on the original images which

can be loaded into segmentation models by opening the images with the file paths through OpenCV. Normally, it operates one image file at a time which is also the one showing in the main window. There are two units to show the size of the images: one is pixel which is used by original images to measure image sizes, the other is point which is used by the user interface to measure image sizes. The two units can be converted through magnification measurement.

Objects can be managed through both object list and trace list. Object list shows the start and the end sections of the object. Trace list shows all the traces for an image in the main window. When users can add traces through different sections, the start and the end sections of the object in the object list can be modified. By using the object list, users can verify the traces for objects and make corrections accordingly.

4.3 Annotation Interface

Annotation can be applied to individual traces, entire objects, and points of interest in the field. When automatic segmentation is performed, it modifies the coordinates of the points but not the names of the annotation tags. after automatic segmentation, in series file and section files, the data structures of the object are not changed, except the coordinates of the data points, because the individual traces of an object become tighter to the boundaries.

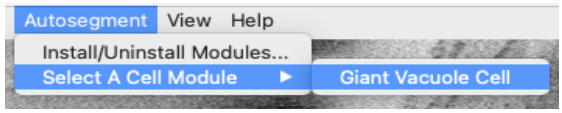
The annotation of the automatic segmentation normally starts with the ROI. However, annotation attributes are customizable. If, by chance, different annotation attributes are selected to draw the ROI, after automatic segmentation, the attributes can be modified to accommodate with those of the other traces for the same object. In short, similar to manual segmentation, the attributes of the traces for individual objects need to be consistent across different sections.

4.4 Automatic Segmentation

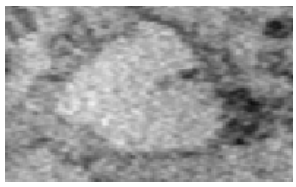
To construct a 3D structure with 2D images, we need to segment GV cells in individual sections and then integrate all the sections together to form a 3D structure. With automatic boundary segmentation, the intermediate steps can be sped up.

To trace a cell boundary through automatic segmentation, we need to follow several steps: (1) Select the Cell Module/Python Package, (2) Locate a GV in a Section, (3) Find the Region Of Interest (ROI), (4) Select Boundary Traces, (5) Auto-segment GV Boundary and Replacing the ROI, (6) Construct the 3D Structure for the GV, as shown in Figure 9.

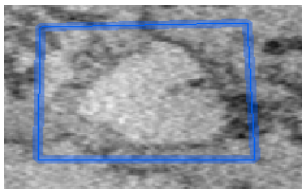
For Step 5 in Figure 9, we use UNet model to perform pixel-level classification. UNet architecture includes encoder, (Semantic and Detail Infusion module (SDI), and decoder. The encoder block extracts low-level features from images, such as boundary information and texture information. The decoder block extracts high-level features, such as the ones which can be used to define the object. SDI integrates encoder and decoder through skip connection mechanism.



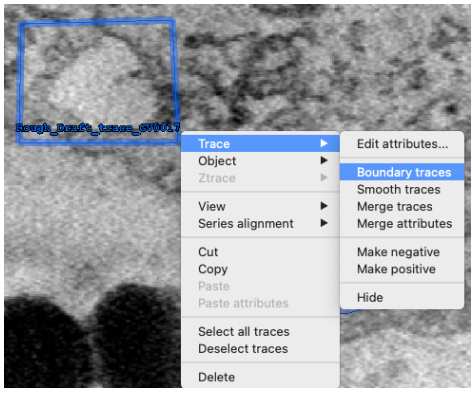
Step 1. Select the Cell Module/Python Package



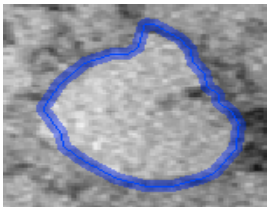
Step 2. Locate a GV in a Section



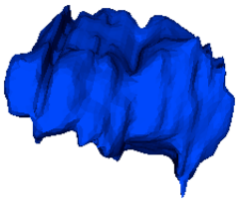
Step 3. Find the Region Of Interest (ROI)



Step 4. Select Boundary Traces



Step 5. Auto-segment GV Boundary and Replacing the ROI



Step 6. Construct the 3D Structure for the GV Cell

Figure 9. Cell Auto-Segmentation and 3D Construction.

UNet is a fully convolution network in which pooling operators are replaced by upsampling operators so that there is no information loss during training and the output presents exactly the original image with unrelated segments removed. In short, UNet can preserve the location information in an image.

This functionality is essential in medical imaging because most of medical imaging tasks require assistance to help read and see more details at pixel level in the medical images, which can not be done by humans.

Because of the image quality, we also use Generative Adversarial Networks (GAN) to perform image enhancement. The idea behind GAN is to learn the data distribution in a supervised manner by assembling the generator and the discriminator. The GAN trains two modules: one is a generator and the other is a discriminator. The way it works is that the discriminator takes the original images as input, the generator starts with random data points, and the distribution of the generator is tuned by that of the discriminator. After training, generator can have the distribution of the discriminator without random noise. Most of the random noise is caused by the technical limitation of the devices, under-exposure/over-exposure, etc.

We connect the two networks together to improve the quality of the results. The two networks are tuned separately. The GAN learns the distribution of the images in a series and generates new images for given image segments in related sections. The UNet uses the enhanced image segments to predict boundaries. We provide pre-trained GAN models with default weights trained on Electron Microscopy images. However, given a new series of images, the pre-trained GAN can be customized by refreshing the weights with new images.

4.5 ROI alignment

To generate 3D structures through 2D images, it gives rise to the following requirements.

1. Each GV segment is a slice of its 3D structure so that all the segments for an individual GV (from the beginning of the 3D structure to the end of the 3D structure) need to be identified in adjacent sections.
2. The photo-shooting limitation in medical imaging needs to be overcome through image enhancement so that the results of automatic segmentation can be affected less by the image quality across different sections.
3. All the ROIs of the object need to be aligned across different sections to achieve consistent results.

A set of SBF-SEM images are generated for an individual object, such as an eye, so that adjacent images are related to each other: the difference in adjacent images is small and the majority is the same. When the ROIs are aligned very well, this feature in SBF-SEM images can be captured.

Automatic segmentation works on the ROIs instead of the entire images. The selected regions need to be manipulated in different ways to make problem-specific data, such as brightness, sharpening, noise removal, etc. Different regions can have different data issues so that the consistent alignment of the ROIs can help improve the data quality and ease the burden of the image correction.

4.6 Data Visualization

Segmentation issues in 2D views are not as noticeable as in 3D views. Segmentation accuracy in 2D views is not as noticeable as in 3D views. when objects are rendered in 3D, the segments of the object are piled together, which showcase the consistency of the object annotation, as well as all the annotation issues, such as misalignment, poor segmentation, misplaced traces, etc. Switching between 2D and 3D views can facilitate the improvement of the annotation.

5. Conclusion

We modified the architecture of Reconstruct, an image annotation tool, which has been generally used in different fields. To facilitate the integration of AI models, we added a menu item – Autosegment, to Reconstruct, present the way to use the attributes of the components and the interactions between the components, and defined the way to manage, integrate and utilize Python package to make predictions. To showcase how useful the new design is, we used a swimming diagram to demonstrate the way to integrate an AI model into Reconstruct. We also implemented a GV cell segmentation task, integrated this segmentation model into Reconstruct, and evaluated the model performance.

References

[1] R. W. Castro, M. C. Lopes, L. M. De Biase, G. Valdez, Aging spinal cord microglia become phenotypically heterogeneous and preferentially target motor neurons and their synapses. Glia 72, 206–221 (2024).

[2] D. Djama et al., The type of inhibition provided by thalamic interneurons alters the input selectivity of thalamocortical neurons. Curr. Res. Neurobiol. 6, 100130 (2024).

[3]. K. Haruwaka et al., Microglia enhance post- anesthesia neuronal activity by shielding inhibitory synapses. Nat. Neurosci. 27, 449–461 (2024).

[4]. D. Holl et al., Distinct origin and region- dependent contribution of stromal fibroblasts to fibrosis following traumatic injury in mice. Nat. Neurosci. 27, 1285–1298 (2024).

[5]. M. K. P. Joyce, J. Wang, H. Barbas, Subgenual and hippocampal pathways in amygdala are set to balance affect and context processing. J. Neurosci. 43, 3061–3080 (2023).

[6]. Y. M. Morozov, P. Rakic, Disorder of golgi apparatus precedes anoxia- induced pathology of mitochondria. Int. J. Mol. Sci. 24, 4432 (2023).

[7]. M. Ziółkowska et al., Phosphorylation of PSD- 95 at serine 73 in dCA1 is required for extinction of contextual fear. PLoS Biol. 21, e3002106 (2023).

[8]. S. Amemiya et al., Early stalked stages in ontogeny of the living isocrinid sea lily Metacrinus rotundus. Acta Zool. 97, 102–116 (2016).

[9]. J. L. Parke et al., Phytophthora ramorum colonizes tanoak xylem and is associated with reduced stem water transport. Phytopathology 97, 1558–1567 (2007).

[10]. C. Groh, Z. Lu, I. A. Meinertzhagen, W. Rössler, Age- related plasticity in the synaptic ultrastructure of neurons in the mushroom body calyx of the adult honeybee Apis mellifera. J. Comp. Neurol. 520, 3509–3527 (2012).

[11]. M. A. Seid, E. Junge, Social isolation and brain development in the ant Camponotus floridanus. Sci. Nat. 103, 1–6 (2016).

[12]. J. Stökl, G. Herzner, Morphology and ultrastructure of the allomone and sex- pheromone producing mandibular gland of the parasitoid wasp Leptopilina heterotoma (Hymenoptera: Figitidae). Arthropod Struct. Dev. 45, 333–340 (2016).

[13]. M. Baumgart et al., Morphometric study of the two fused primary ossification centers of the clavicle in the human fetus. Surg. Radiol. Anat. 38, 937–945 (2016).

[14]. B. C. Moore, K. Mathavan, L. J. Guillette, Morphology and histochemistry of juvenile male American alligator (Alligator mississippiensis) phallus. Anat. Rec. 295, 328–337 (2012).

[15]. C. M. Dinnis, A. K. Dahle, J. A. Taylor, Three- dimensional analysis of eutectic grains in hypoeutectic Al–Si alloys. Mater. Sci. Eng. A, Struct. Mater. Prop. Microstruct. Process. 392, 440–448 (2005).

[16] Grierson, I., Lee, W.R., 1975. Pressure-induced changes in the ultrastructure of the endothelium lining Schlemm's canal. Am. J. Ophthalmol. 80, 863–884.

[17] Johnstone, M., Grant, W.M., 1973. Pressure-dependent changes in structures of the aqueous outflow system of human and monkey eyes. Am. J. Ophthalmol. 75, 365–383.

[18] Tripathi, R.C., 1972. Aqueous outflow pathway in normal and glaucomatous eyes. Br. J. Ophthalmol. 56, 157–174.
[19] Allingham, R.R., de Kater, A.W., Ethier, C.R., Anderson, P.J., Hertzmark, E., Epstein, D. L., 1992. The relationship between pore density and outflow facility in human eyes. Invest. Ophthalmol. Vis. Sci. 33, 1661–1669.
[20] Johnson, M., Chan, D., Read, A.T., Christensen, C., Sit, A., Ethier, C.R., 2002. The pore density in the inner wall endothelium of Schlemm's canal of glaucomatous eyes. Invest. Ophthalmol. Vis. Sci. 43, 2950–2955.